

Final Project Report

ML for Network Traffic Anomaly Detection

Arpan Kapoor

EE 467 - Machine Learning for Cybersecurity

Problem Description.....	2
Threat Model.....	2
Dataset and Data Characteristics.....	3
Exploratory Data Analysis.....	4
Data Preprocessing and Feature Engineering.....	6
Machine Learning Pipeline.....	7
Models Used.....	8
Experimental Setup and Evaluation Methodology.....	9
Results.....	10
Interpretation, Conclusion, and Future Work.....	14
Team Contributions.....	15
References.....	15

Problem Description

This final project focuses on the area of network intrusion detection. The goal is to identify malicious network traces that are nested within large amounts of benign data. Network intrusion attacks can target sensitive data, companies' services, and critical infrastructure. This is especially important because many of these network intrusions are considered "zero day" attacks, meaning that the intrusion is deployed so early that the victim company or organization has had no time to react and update their detection systems. Thus, tracking down and identifying these intrusions is very valuable. Finding a viable solution is important because cyber attacks compromise sensitive data, disrupt infrastructure, and cause massive financial damage that can affect millions of people. Organizations need adaptive systems that can detect novel threats because not only are the organizations themselves affected, but also the users who rely on their utilities, services, and most importantly, the security of their data.

Threat Model

Some examples of network intrusion attacks include DDoS, Bot, PortScan, SQL Injection, and Web Attacks. These attacks manipulate network attributes such as packet lengths, window sizes, and subflow bytes. They are often designed to mimic normal traffic in order to go undetected using tactics such as spoofed IP addresses. Additionally, normal traffic regularly changes, so static defensive measures are inadequate. Traditional network intrusion detection systems such as Snort and Suricata rely on static, manually created signatures or rules. What this essentially means is if the detection system finds the existence of a certain bit sequence or data pattern for example, it will flag the trace as malicious. While this can be fast, they are blind to zero day attacks. They are also easily evaded by minor behavioral modifications and fail to adapt to shifting network traffic. Furthermore, security operations personnel can face "alert fatigue" when systems generate too many false positives. This is why machine learning can be a more efficient approach; it can learn to generalize patterns in the data, and to detect anomalies in the form of zero day attacks. Supervised models can learn complex attack patterns, while unsupervised models can learn the geometric boundary of normal traffic to flag anomalies.

Dataset and Data Characteristics

The dataset used for this project is the [CIC-IDS2017](#) Intrusion Detection Evaluation Dataset (GeneratedLabelFlows ver.), representing aggregate network traffic flow data without any preliminary cleaning. The reason this version was chosen as opposed to the machine learning version was so that all preprocessing and cleaning steps could be performed manually, resulting in more accurate control of the data.

Each row in this dataset represents a summarized bidirectional network flow connection, with relevant statistical metadata instead of raw packet information. The dataset contains around 85 mostly numerical features such as the packet length, source and destination ports, etc. It contains around 2.3 million entries. The dataset contains multiple specific attack labels and a BENIGN label. To build a generalized intrusion detection system and to make the problem simpler to approach, the labels were binarized, where 1 = malicious and 0 = benign. This created a somewhat imbalanced dataset, with roughly 76% representing normal traffic and 24% representing malicious traffic. This class imbalance requires altered model parameters and evaluation metrics in order to prevent the models from trivially predicting the majority class. The dataset additionally contains a timestamp feature that represents the time of the network connection. These were used to order the data chronologically for a time based split, and were removed after this in order to prevent time-based cheat predictions.

Exploratory Data Analysis

With 85 numerical features, the dataset suffers from the large dimensionality issues, making it difficult to visualize directly and leading to some models struggling to process without noise.



Figure 1: Principal Component Analysis Results

Principal Component Analysis (PCA) was used to capture global variance, projecting the 85 dimensions down to 2. The results, as shown in Figure 1, revealed that a considerable amount of the malicious traffic cluster (in blue) overlaps with the core of the normal traffic (in orange). The PCA results however did not do a great job at showing the finer details of this overlap. To better understand the specifics of the overlap and imbalance issues with the data, a t-Distributed Stochastic Neighbor Embedding (t-SNE) was applied on a 10,000 point sample of the scaled test data.

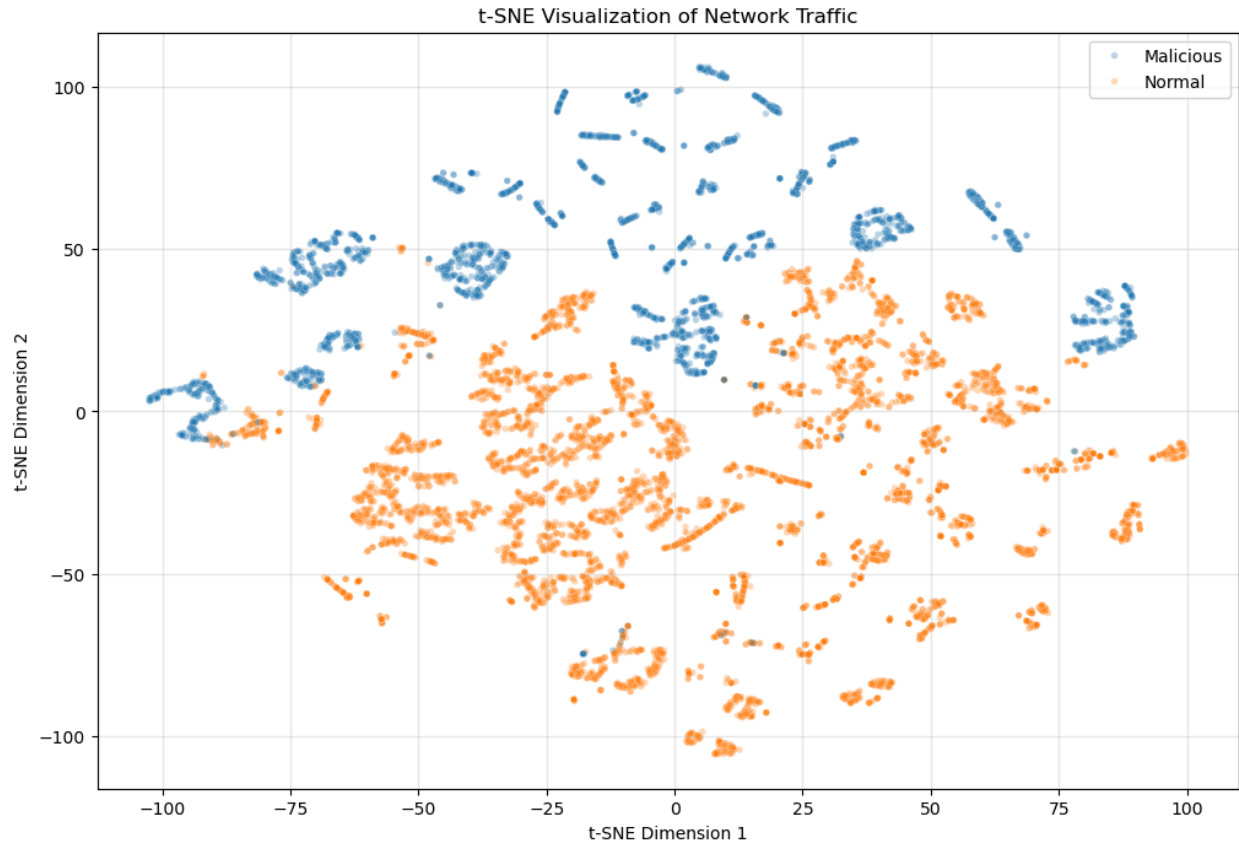


Figure 2: t-Distributed Stochastic Neighbor Embedding Results

Unlike PCA's linear projection, the t-SNE results (Figure 2) revealed distinct, non-linear islands of malicious traffic separate from the benign traffic, while also showing the overlapping regions with more specificity. This confirmed that while the data is messy in terms of overlap, attacks still have distinct characteristics that non-linear models can identify. This visualization also illustrated the data overlap with more granularity, demonstrating how advanced attacks disguise themselves as benign traffic to remain undetected by traditional rule or signature based detection systems.

Data Preprocessing and Feature Engineering

This dataset comes as seven separate datafiles representing each day of the week. These files were globbed together to construct one dataset representing the entire week's network flow history. As previously mentioned, the attack labels describing each type of attack (or if the connection was benign) were binarized to represent either a malicious or benign connection. Additionally, after converting the timestamp values to usable datetime values, the data was chronologically sorted before splitting. This simulates realistic conditions where a model is trained on past behavior and tested on future traffic that has not been seen before. Column names were cleaned and normalized to remove any unnecessary whitespace or formatting, and rows containing infinite values (`np.inf`) and null values (`NaN`) were dropped. This ensured a clean and error-free dataset during training. Non-predictive features such as `source_ip`, `destination_ip`, `flow_id`, and `timestamp` were dropped, as well as any non-numeric features. This ensured that the models learned the behavioral patterns of an attack instead of memorizing cheat metrics such as attacker IPs or timestamps associated with attacks.

To counter the class imbalance within the data and to avoid bias towards the majority benign class, the `class_weight='balanced'` parameter was used for the supervised models, penalizing them for misclassifying the minority malicious class. This does so by weighing the minority class heavier during training, applying a larger penalty for misclassifying malicious connections and forcing the model to learn the attack patterns instead of falling back to the majority prediction. `StandardScaler` was applied to normalize the remaining features, which was a requirement for the distance-based algorithms such as `LogisticRegression` and `OneClassSVM` as well as for the dimensionality reduction techniques used like PCA and t-SNE mentioned previously.

Machine Learning Pipeline

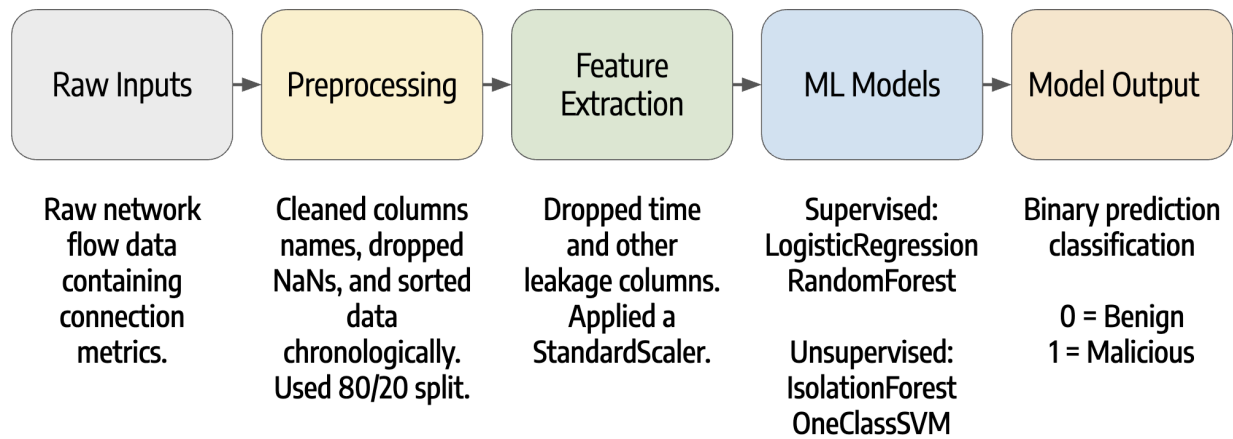


Figure 3: Machine Learning Pipeline Diagram

The **raw input** consisted of a handful of CSVs of network flow data representing traffic from each day of the week. As previously mentioned, this was the truly raw version of the data from the CIC to avoid any unwanted preliminary cleaning steps already applied by them. As discussed in greater detail in the previous section, the **preprocessing** stage included steps for normalizing column names, chronological sorting, binarization of the malicious type, removal of missing values, as well as applying a standard 80/20 split for training and testing data. For **feature extraction**, any potential leakage columns such as IP information and timestamp were dropped from the training and testing sets. A `StandardScaler` was applied to normalize data ranges for the use in models relying on distance-based calculations. The **models** utilized in this project included two supervised models: `LogisticRegression` and `RandomForest`, and two unsupervised models: `IsolationForest` and `OneClassSVM`. The **output** of this pipeline consisted of a binary prediction classification of malicious or benign enforced on a custom false positive rate (FPR) threshold of 1%.

Models Used

As highlighted in the pipeline section, **LogisticRegression** was used as a linear, supervised baseline. As previously mentioned, both supervised models used hyperparameter `class_weight='balanced'` to counter the data imbalance. Additional parameters included `max_iter=1000` and `random_state=42` (which all applicable models used).

The other supervised model utilized was **RandomForest**, a tree-based model chosen to handle high dimension feature spaces and capture complex non-linear attack patterns without being affected by the large scale of the data. The parameters used included `n_estimators=100` (the default value) for sufficient complexity without overly excessive resource usage, and `n_jobs=-1` to utilize all available CPUs. This was the first model employed after the baseline, with the hope of handling the large data variety more efficiently than `LogisticRegression`.

The first unsupervised model utilized was **IsolationForest**, another tree-based model chosen to see if attacks could be isolated based purely on being rare and isolated. The parameters used included `n_estimators=300`, and `contamination='auto'` which automated the threshold at which the decision function classified points as outliers. The hope with this model was to do a better job at identifying the more unique attacks that occurred later in the week which the model had not previously seen during training. As expected though, this model struggled with detecting the overlapping malicious values.

The second unsupervised model utilized was **OneClassSVM**, a distance-based model used to see if geometric boundaries around normal training data could lead to successfully flagging attacks. The `nu` parameter was optimized to 0.2 to allow a wider, softer boundary that accounts for training data noise. This was determined by running multiple tests with varying `nu` values ranging from 0.1 to 0.5, with the best results coming from the parameter set to 0.2. This was the final model tested, with the hope of finding a middle ground between detecting newer, unseen attacks, and still having some sort of success at differentiating the overlapping data.

Experimental Setup and Evaluation Methodology

As mentioned in the pipeline section, an 80/20 chronological split was utilized for the training and testing process. To correctly test the models' resilience against data drift, random cross validation was intentionally avoided. Because the plan was to use a time-based split, traditional train test split functions were not usable since they split data randomly. Thus, the data was split manually by first organizing the data chronologically, identifying the index of the value at the 80% mark in the data, and then using that index to execute the split.

The primary evaluation metric was recall at a strict 1% false positive rate. Because of the class imbalance, standard accuracy scores can be a misleading metric. In a real network intrusion detection system, a model with even a 5% FPR could mean millions of false alarms, leading to the issues previously discussed. A naive model predicting just benign traffic could achieve 90% accuracy while still failing to detect a single malicious threat and inducing issues of alert fatigue as discussed previously. Therefore, the models were evaluated at a threshold that capped false positives at 1% (as well as 5% to analyze the jump in performance between these two thresholds), and then measuring how many actual attacks could be caught under that constraint. This ensures that the models' performance was judged only on its ability to identify the minority class without creating an unacceptable amount of false alarms.

Results

```
LogisticRegression Results at 1.0% FPR:  
Actual FPR: 1.000%  
Recall: 63.870%  
Threshold: 86.688%  
  
LogisticRegression Results at 5.0% FPR:  
Actual FPR: 4.999%  
Recall: 78.070%  
Threshold: 66.876%
```

Figure 4: LogisticRegression Results

Surprisingly, the baseline `LogisticRegression` performed decently well, with a recall of 63.870% at a forced 1% FPR as shown in Figure 4. At 5% FPR, the recall jumps up to just over 78%. This was pleasantly surprising considering the complex nature of the dataset, and increased the expectations for the upcoming models.

```
RandomForestClassifier Results at 1.0% FPR:  
Actual FPR: 0.945%  
Recall: 67.307%  
Threshold: 16.000%  
  
RandomForestClassifier Results at 5.0% FPR:  
Actual FPR: 4.998%  
Recall: 89.018%  
Threshold: 2.000%
```

Figure 5: RandomForest Results

As shown in Figure 5, `RandomForest` performed slightly better than the `LogisticRegression`, with 67.307% recall at 1% FPR. There was also a much larger jump up to just over 89% at a 5% FPR. Another interesting insight was that the `RandomForest` model had a very high precision rate, highest out of all the models tested in fact. Although this wasn't the main evaluation metric, it means the model did well at minimizing false positives. Even though this can help solve the issue of alert fatigue, it would still be more beneficial to do

better at minimizing false negatives, preventing any malicious connections from going undetected. This was the hopeful goal of `IsolationForest`.

```
IsolationForest Results at 1.0% FPR:
Actual FPR: 0.999%
Recall: 0.000%
Threshold: 20.727%

IsolationForest Results at 5.0% FPR:
Actual FPR: 4.999%
Recall: 14.483%
Threshold: 9.445%
```

Figure 6: IsolationForest Results

As seen in Figure 6 though, `IsolationForest` failed massively with a recall of 0% at 1% FPR, and only a slight increase to almost 15% at 5% FPR. Not only did this model fail to properly isolate any new, unseen attacks, but it also failed at capturing the previous attack patterns the other two models did so well with, indicating its struggle with chronological data drift. The results of this model do a good job at emphasizing the struggle of handling data drift in these complex cybersecurity settings, as well as the effectiveness of zero day attacks.

```
OneClassSVM Results at 1.0% FPR:
Actual FPR: 0.999%
Recall: 54.656%
Threshold: 69.354%

OneClassSVM Results at 5.0% FPR:
Actual FPR: 4.997%
Recall: 74.251%
Threshold: 13.225%
```

Figure 7: OneClassSVM Results

With the hopes of reviving the success of this detection system, one last model was used to ideally find a middle ground between solving the two issues of data drift and zero day attacks.

OneClassSVM performed decently, with a recall of 54.656% at 1% FPR and nearly 75% at 5% FPR, though not nearly as well as the first two models. As previously mentioned, after numerous trial runs of this model, the nu parameter of 0.2 was found to be the value that achieved the best results. Although this model did not perform as well as the first two, it showed that there was still potential in the form of unsupervised models, despite the failure of the other unsupervised IsolationForest.

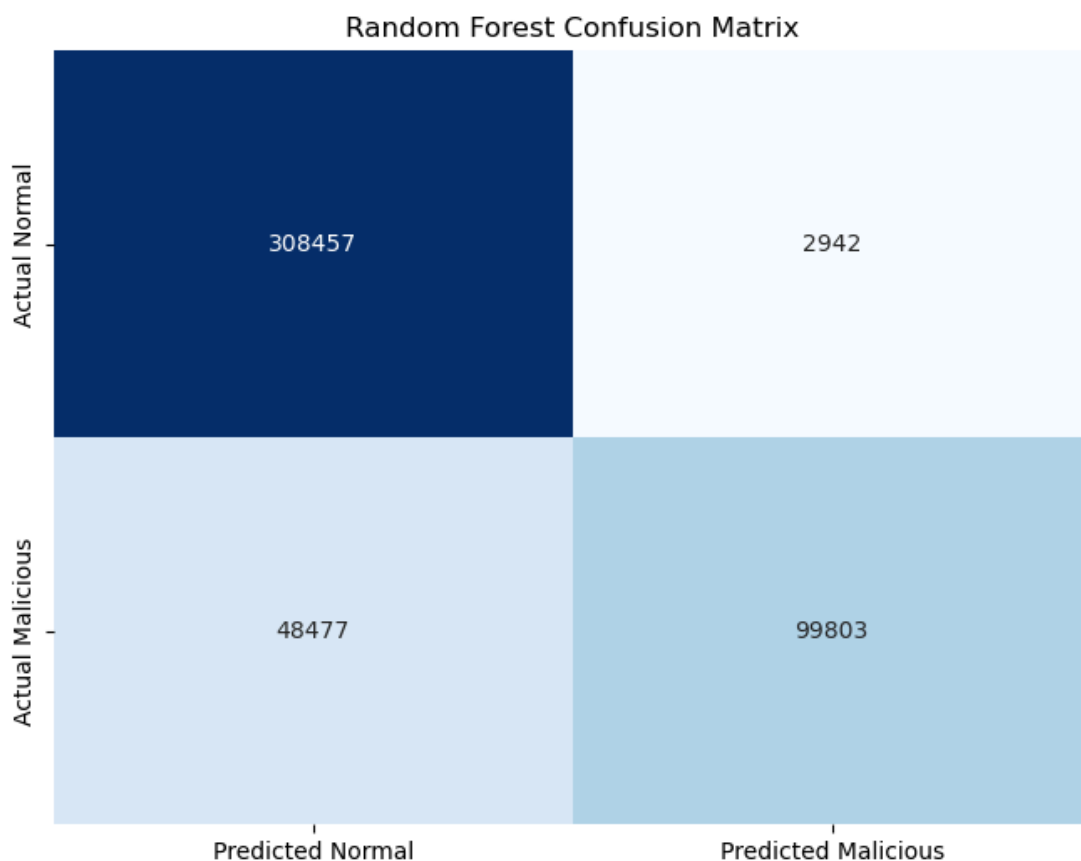


Figure 8: RandomForest Confusion Matrix

After analyzing each of the models' recall results at 1% FPR, it is clear that RandomForest performed the best overall. This is reinforced by the confusion matrix in Figure 8 compared to the results of the other models, though it also highlights that a lot of work still needs to be done in order to correctly classify the remaining large amounts of undetected malicious connections that are likely doing well at disguising and overlapping as normal traffic, with a mixture of zero day attacks.

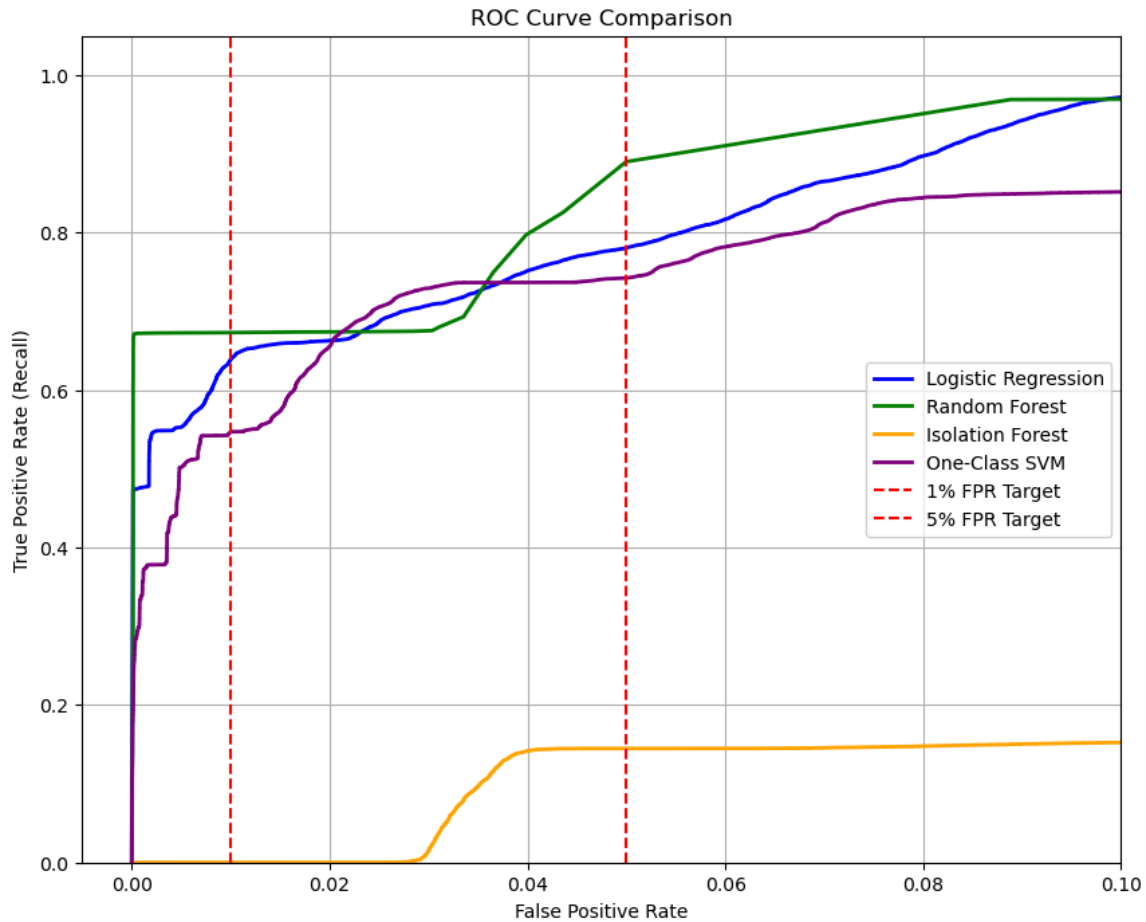


Figure 9: ROC Curve Comparison Plot

As seen in the ROC curve comparison in Figure 9, `RandomForest` consistently leads in terms of minimizing recall for the majority of the forced false positive rate thresholds, with the 1% and 5% thresholds marked in red. The graph also highlights that the `LogisticRegression` and `OneClassSVM` models were not far behind and definitely serve as viable options with the potential of further tweaking and optimization. As with the conclusion of the `IsolationForest` results, the rest of the models are still not perfect. The presence of zero day attacks and data drift will always persist, but employing methods that are more reactive such as machine learning to combat this kind of cybersecurity issue is key. While they were unable to absolutely predict all values, there is still much more room for improvement in terms of hyperparameter optimization. Optimistically, models such as `RandomForest` showed promising results with minimal altering.

Interpretation, Conclusion, and Future Work

The (relative) success of `RandomForest` and the failure of `IsolationForest` highlight the fundamental challenge in network intrusion detection: normal network traffic is constantly evolving. Unsupervised detection models struggle when the baseline of benign traffic shifts over time. While supervised models do better at handling this phenomenon, they struggle with their own issue of zero day attacks. When these two issues collaborate, which they often do, both supervised and unsupervised models tend to struggle.

Supervised, nonlinear models like `RandomForest` are highly effective when attack signatures are present in the training data. Distance-based unsupervised models like `OneClassSVM` can be a good alternative for detecting zero day attacks as long as their boundaries are properly tuned to absorb data drift. The time-based split exposed the limitations of models trained on past data, with them struggling to identify patterns in newer, unseen data. Furthermore, the overlap in the high dimensional space indicates that complex attacks successfully blend in with benign traffic patterns.

Future work should explore continuous learning, such as retraining the baseline retroactively as new data becomes available each day to combat data drift. Additionally, hybrid ensemble methods where an unsupervised model flags novel anomalies for a supervised model to categorize could combine the benefits of the individual models tested in this project. For example, a `OneClassSVM` could be paired with a `RandomForest` to hopefully do a better job at tackling the combined issue of data drift and zero day attacks highlighted throughout this report.

Team Contributions

As I worked on my own, I was responsible for all relevant tasks, including but not limited to cleaning, feature engineering, model curation and tuning, evaluation, and reporting.

References

The dataset used for this project, as linked and described in detail in the Dataset and Data Characteristics section, was provided by the Canadian Institute of Cybersecurity at the University of New Brunswick. It is free to use and provided for download on the linked webpage. It requires an email signup/verification to download.

The following libraries, frameworks, and/or tools were used for this project:

```
matplotlib.pyplot, seaborn, pandas, numpy, glob,  
LogisticRegression, SGDOneClassSVM from sklearn.linear_model  
StandardScaler from sklearn.preprocessing  
roc_curve, classification_report, confusion_matrix from  
sklearn.metrics  
RandomForestClassifier, IsolationForest from sklearn.ensemble  
PCA from sklearn.decomposition  
TSNE from sklearn.manifold
```